

CCNA 200-301 V2.0

24 Access Control Lists

Part IV — Network Services and Security

EXAM TOPICS

4.6

LECTURER

Dr. Mustafa Hameed

THE ISLAMIA UNIVERSITY OF BAHAWALPUR

Department of Information Technology

What you will be able to do

- **Calculate** a wildcard mask for any subnet, and recognise the common ones on sight.
- **Configure** standard, extended, numbered and named IPv4 ACLs.
- **Decide** where to place an ACL and in which direction, and justify the choice.
- **Read** an existing ACL and predict exactly which traffic it permits — which is what the exam asks far more often than it asks you to write one.

Before we start

Chapter 4 for subnet masks — a wildcard mask is meaningless without them. Chapter 6 for protocol numbers and well-known ports. Chapter 22 for the order of operations, which decides which address an ACL actually sees.

Vocabulary for this chapter

access control list

wildcard mask

implicit deny

standard ACL

extended ACL

named ACL

sequence number

ip access-group

access-class

established

Each is defined where it first matters — not here.

IPv4 only, and all four kinds

This is what the blueprint actually asks for

Topic **4.6** reads *configure IPv4 access control lists (standard, extended, numbered, and named ACLs)*. IPv6 ACLs are not on this blueprint.

All four named kinds are examinable, but note they are two axes, not four categories: an ACL is standard *or* extended by what it can match, and numbered *or* named by how you refer to it. All four combinations exist.

Access control list

An ordered list of `permit` and `deny` statements. Each packet is tested against the entries from the top down; the **first match wins** and testing stops. A packet matching nothing hits an invisible `deny any` at the end and is dropped.

Wildcard for 172.16.8.0/21

The mask for a /21 is 255.255.248.0.

Subtract each octet from 255: 255-255=0, 255-255=0, 255-248=7, 255-0=255.

The wildcard is 0.0.7.255, and the ACL entry is `permit ip 172.16.8.0 0.0.7.255 any`. It matches 172.16.8.0 through 172.16.15.255 — eight consecutive /24s, which is what a /21 is.

The check that catches arithmetic slips

Add the wildcard to the network address. The result should be the last address in the range you meant. $172.16.8.0 + 0.0.7.255 = 172.16.15.255$. If that is not the broadcast address of the block you intended, the wildcard is wrong.

Do this every time. It takes five seconds and it catches the off-by-one that otherwise costs you a lab.

A numbered standard ACL

IOS · configuration mode

```
! Order matters: the specific deny must precede the general permit,  
! or the permit matches first and the deny is never reached.  
access-list 10 remark Block the lab subnet, allow the rest of campus  
access-list 10 deny    192.168.30.0 0.0.0.255  
access-list 10 permit 192.168.0.0 0.0.255.255  
!  
interface GigabitEthernet0/0/1  
  ip access-group 10 out
```

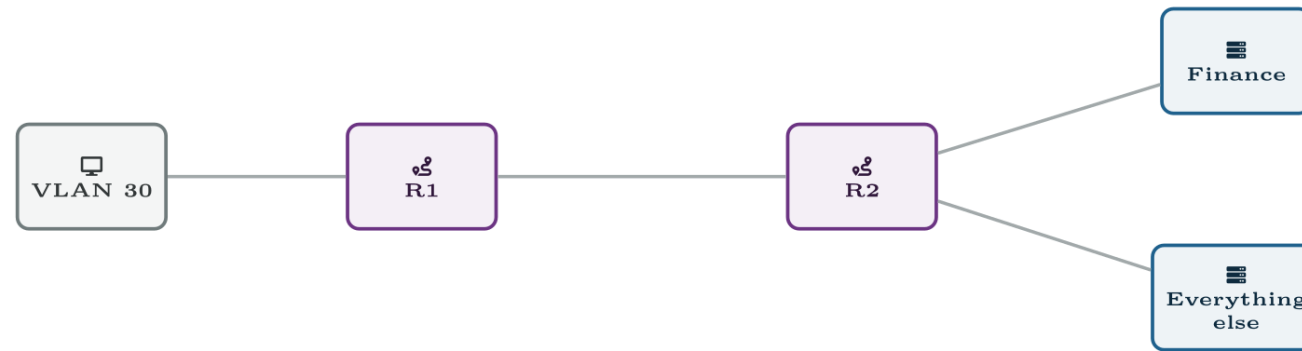
Place a standard ACL close to the destination

Because it can only see the source address, a standard ACL cannot distinguish “traffic from this subnet to the finance server” from “traffic from this subnet to anywhere”. Applied near the source, it blocks the host from the entire network, not just from the thing you wanted protected.

So the rule is: standard ACLs go as close to the destination as possible; extended ACLs go as close to the source as possible, because they can be precise enough to be safe there and it saves carrying doomed traffic across the network.

Standard ACLs

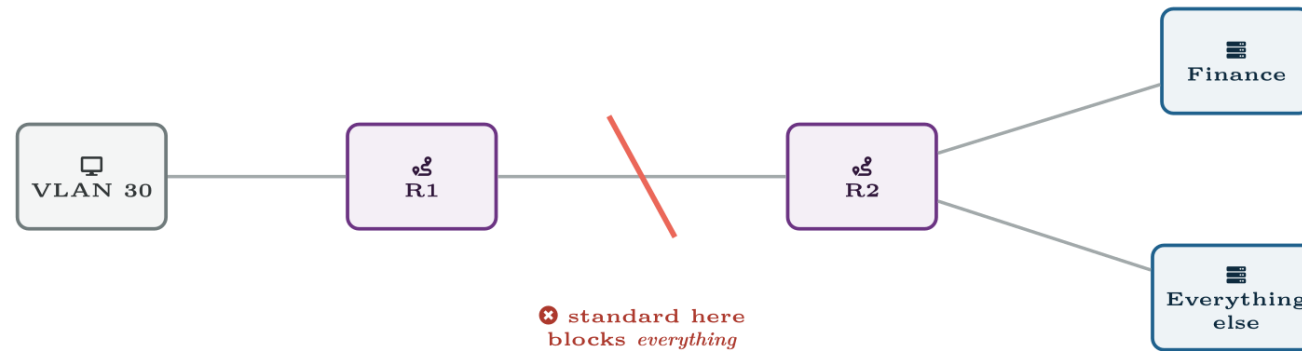
VLAN 30 must not reach the finance server — but must still reach everything else



Two servers, one path. Whatever we write has to separate them.

Standard ACLs

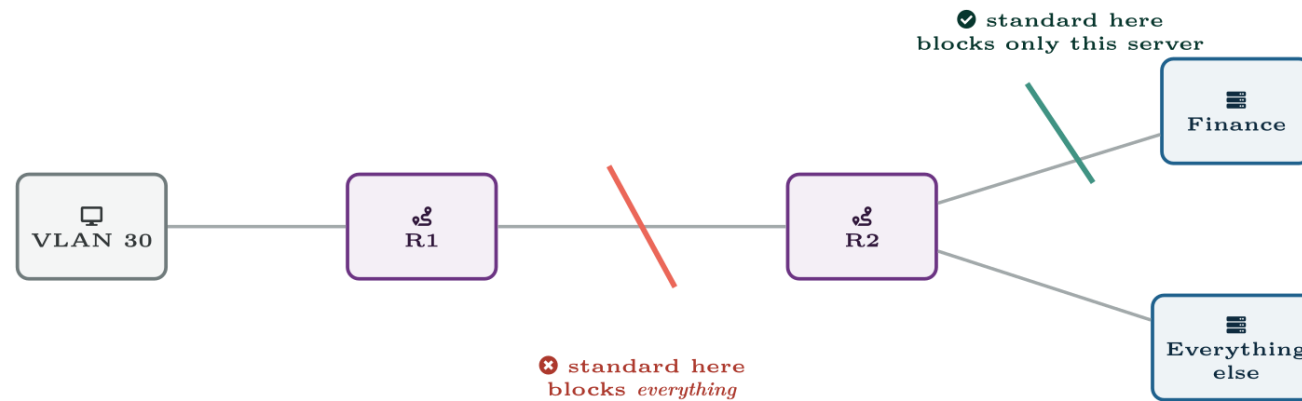
VLAN 30 must not reach the finance server — but must still reach everything else



Near the source looks right — it stops doomed traffic early. But a standard ACL matches only *who sent it*, and here both servers are behind the same next hop. Everything VLAN 30 sends is dropped.

Standard ACLs

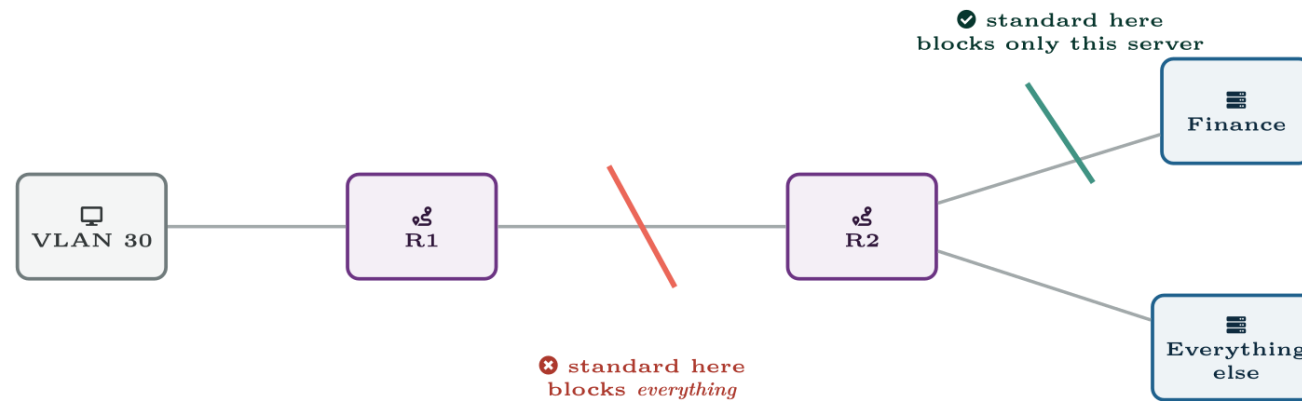
VLAN 30 must not reach the finance server — but must still reach everything else



Move it to the last hop and the two servers are finally on different interfaces, so matching the source alone is enough.

Standard ACLs

VLAN 30 must not reach the finance server — but must still reach everything else



Hence the rule: a standard ACL goes near the destination, because that is the only place its one match is sufficient. An extended ACL names the destination too, so it is safe at the source — and saves carrying doomed traffic across the network.

An extended ACL letting a subnet reach only what it needs

IOS · configuration mode

```
access-list 110 remark Guest VLAN -- web, DNS and nothing else
!
! DNS to the campus resolvers, both transports.
access-list 110 permit udp 192.168.40.0 0.0.0.255 host 192.168.20.10 eq 53
access-list 110 permit tcp 192.168.40.0 0.0.0.255 host 192.168.20.10 eq 53
!
! Web out to anywhere.
access-list 110 permit tcp 192.168.40.0 0.0.0.255 any eq 80
access-list 110 permit tcp 192.168.40.0 0.0.0.255 any eq 443
!
! Let guests get an address. Without this the implicit deny stops
! DHCP and the VLAN never works at all.
access-list 110 permit udp any host 255.255.255.255 eq 67
!
! Ping to the gateway only, so users can prove their own link.
access-list 110 permit icmp 192.168.40.0 0.0.0.255 host 192.168.40.1
!
! Everything else is dropped by the implicit deny, logged so that
! surprises are visible.
access-list 110 deny ip any any log
```

An extended ACL letting a subnet reach only what it needs (continued)

IOS · configuration mode

```
!  
interface GigabitEthernet0/0/2  
  description Guest VLAN  
  ip access-group 110 in
```


eq 53 on the destination, not the source

In that list the port qualifier follows the *destination* address, because the guest is connecting *to* port 53 on the resolver. Its own source port is ephemeral and unpredictable, so it is left unspecified.

Getting this backwards — `permit udp 192.168.40.0 0.0.0.255 eq 53 any` — is a classic and produces an ACL that matches almost nothing. The position of the port in the line is the whole meaning.

established

This is what the blueprint actually asks for

`permit tcp any 192.168.10.0 0.0.0.255 established` matches only TCP segments with the ACK or RST flag set — that is, packets belonging to a conversation that somebody inside already started. A pure connection attempt from outside has only SYN set and is not matched.

It is a cheap way to say “replies to our traffic may return, new connections may not.” It is not stateful — it inspects flags, not connections — and the exam likes that distinction.

The same policy, named and editable

IOS · configuration mode

```
ip access-list extended GUEST-OUT
  remark Guest VLAN -- web, DNS and nothing else
  10 permit udp 192.168.40.0 0.0.0.255 host 192.168.20.10 eq 53
  20 permit tcp 192.168.40.0 0.0.0.255 host 192.168.20.10 eq 53
  30 permit tcp 192.168.40.0 0.0.0.255 any eq 80
  40 permit tcp 192.168.40.0 0.0.0.255 any eq 443
  50 deny ip any any log
!
interface GigabitEthernet0/0/2
  ip access-group GUEST-OUT in
```

This is the practical advantage, and it is a large one

With sequence numbers you can insert and delete single lines:

- `ip access-list extended GUEST-OUT then 35 permit tcp ... eq 8080` inserts a rule between 30 and 40.
- `no 40` removes one entry and leaves the rest standing.

With a numbered ACL on older IOS, `no access-list 110` deletes the *entire list* — and if it is still applied to an interface, the interface now has an empty ACL, which denies everything. That is how a routine edit becomes an outage. Number your entries in tens so there is room to insert later, and prefer named ACLs.

Restricting management access

IOS · configuration mode

```
ip access-list standard MGMT-HOSTS
 10 permit 10.0.0.0 0.0.0.255
 20 permit host 192.168.99.14
!
line vty 0 15
 access-class MGMT-HOSTS in
 transport input ssh
```

show access-lists

R1#

Standard IP access list MGMT-HOSTS

10 permit 10.0.0.0, wildcard bits 0.0.0.255 (214 matches)

20 permit 192.168.99.14 (3 matches)

Extended IP access list GUEST-OUT

10 permit udp 192.168.40.0 0.0.0.255 host 192.168.20.10 eq domain (8841 matches)

20 permit tcp 192.168.40.0 0.0.0.255 host 192.168.20.10 eq domain (12 matches)

30 permit tcp 192.168.40.0 0.0.0.255 any eq www (40218 matches)

40 permit tcp 192.168.40.0 0.0.0.255 any eq 443 (338104 matches)

50 deny ip any any log (2277 matches)

The match counters are the diagnostic

That output tells you more than the configuration does.

- **A rule with zero matches** is either unnecessary or unreachable — shadowed by an earlier line. Both are worth investigating.
- **A rising counter on the final deny** tells you the ACL is dropping things, and `log` tells you what. This is the first place to look when an application “sometimes” fails.
- **Counters that do not move at all** mean the ACL is not in the path. Check it is actually applied, and in the direction you think.

`clear access-list counters` before a test gives you a clean baseline.

show ip interface GigabitEthernet0/0/2 | include access list

R1#

```
Outgoing access list is not set  
Inbound  access list is GUEST-OUT
```


Common pitfalls

- **Forgetting the implicit deny.** Permit what you need and everything else stops. This is the single biggest cause of ACL outages.
- **A general permit above a specific deny.** First match wins, so the deny never runs.
- **A standard ACL applied near the source.** Blocks the host from everything, not from the one destination you meant.
- **Port qualifier on the wrong side.** `eq 80` belongs with the destination for a client reaching a web server.
- **Writing the ACL and not applying it.** No error, no effect.

Common pitfalls (continued)

- **Applying it in the wrong direction.** `in` and `out` are relative to the interface, not to the building.
- **`no access-list 110` on a live numbered ACL.** Deletes the whole list; the interface is left denying everything.
- **Blocking your own management session.** Think about `vty` before you press enter on a remote device.
- **Forgetting supporting protocols.** DHCP, DNS and ICMP all need explicit permits in a restrictive list, or the VLAN looks completely dead.

An ACL meant to stop one subnet reaching the finance server has cut that subnet off from everything.

The requirement was: hosts in **192.168.30.0/24** must not reach the finance server at **192.168.20.15**. Everything else should be unaffected. After the change, VLAN 30 cannot reach the internet, the file server, or DNS. It can still reach hosts within its own VLAN.

An ACL meant to stop one subnet reaching the finance server has cut that subnet off from everything.

What would you check first — and what do you expect to see?

show access-lists 20

R1#

```
Standard IP access list 20
 10 deny    192.168.30.0, wildcard bits 0.0.0.255 (18442 matches)
 20 permit any (0 matches)
```

show ip interface Vlan30 | include access list

R1#

```
Inbound access list is 20  
Outgoing access list is not set
```

What is the fault — and what does this output rule out?

Why it is doing that

Diagnosis. Two things are wrong and they compound.

First, this is a *standard* ACL, so it can only match the source. It has no way to express “to the finance server” — the destination is invisible to it.

Second, it is applied **inbound on VLAN 30’s own SVI**, which is as close to the source as it is possible to get. Every packet leaving VLAN 30 for anywhere hits line 10 and is dropped. Traffic *within* the VLAN still works because it is switched and never reaches the SVI at all — which is exactly why the symptom looks like a routing fault rather than an ACL.

The confirming detail is line 20: `permit any` has **zero matches**. Not one packet from anywhere got past line 10. An ACL whose last line never matches is dropping everything it sees.

An ACL that expresses the actual requirement

IOS · configuration mode

```
ip access-list extended VLAN30-POLICY
 10 remark Deny only the finance server, permit the rest
 20 deny   ip 192.168.30.0 0.0.0.255 host 192.168.20.15
 30 permit ip any any
!
interface Vlan30
 no ip access-group 20 in
 ip access-group VLAN30-POLICY in
```

And the lesson

Fix. The requirement names a destination, so it needs an extended ACL, and an extended ACL can safely sit near the source.

Note line 30. Without it the extended ACL would reproduce the original outage by a different route. Then `clear access-list counters` and re-test: line 20 should accumulate matches only when someone tries the finance server, and line 30 should carry everything else.

Writing, reading and repairing ACLs

Topology. R1 routing four VLANs — staff 10, servers 20, lab 30, guest 40 — with a server at 192.168.20.10 running DNS and HTTP, a finance server at 192.168.20.15, and an internet-facing link.

1. Confirm full connectivity between all four VLANs before applying any ACL. Record the tests you used; you will repeat them after every change.
2. Write and apply a numbered standard ACL restricting management access to the VTY lines to the staff VLAN only. Test from staff and from lab.
3. Convert it to a named ACL. Insert an additional permitted host between two existing lines without retyping the list.
4. Write an extended named ACL for the guest VLAN permitting only DNS to 192.168.20.10, HTTP and HTTPS anywhere, and DHCP. Apply it inbound. Verify each permitted service works and at least three others do not.

Writing, reading and repairing ACLs (continued)

1. Read the match counters after your tests. Identify any rule with zero matches and explain why.
2. **Break 1.** Remove the DHCP permit and release a guest host's lease. Predict the result before testing.
3. **Break 2.** Move the `permit tcp ... eq 443` line above the `deny` that follows it, then below it, and observe the counters in each case.
4. **Break 3.** Reproduce the Break & Fix above deliberately: use a standard ACL near the source to try to protect one server. Confirm the symptom, then repair it with an extended ACL.

Writing, reading and repairing ACLs (continued)

- 1. Break 4.** Apply an ACL that blocks your own SSH session on a device you can still reach by console. Recover it, then write down the habit that would have prevented it.
- 2. Extension.** Add `permit tcp any 192.168.20.0 0.0.0.255 established` to an internet-facing inbound ACL. Verify that internal hosts can browse out and that inbound connections are refused, and explain why this is not a stateful firewall.

CHECKPOINT 1 of 5

Give the wildcard mask for $10.20.16.0/20$ and the range it matches.

Discuss · then advance

Checkpoint 1 of 5

Give the wildcard mask for 10.20.16.0/20 and the range it matches.

Mask 255.255.240.0, so the wildcard is 0.0.15.255. It matches 10.20.16.0 through 10.20.31.255 — sixteen consecutive /24s.

CHECKPOINT 2 of 5

Why is a standard ACL placed near the destination and an extended ACL near the source?

Discuss · then advance

Checkpoint 2 of 5

Why is a standard ACL placed near the destination and an extended ACL near the source?

Because a standard ACL can only match the *source*. Placed near the source it would block that host from everything, not just the one destination. An extended ACL can name the destination too, so it is precise enough to sit near the source, which saves carrying doomed traffic across the network.

CHECKPOINT 3 of 5

An ACL's final permit any shows zero matches. What does that tell you?

Discuss · then advance

Checkpoint 3 of 5

An ACL's final permit any shows zero matches. What does that tell you?

That everything is being dropped by an earlier line — no packet from anywhere reached the bottom of the list. An ACL whose final permit never matches is denying all traffic it sees.

CHECKPOINT 4 of 5

What does established match, and why is it not stateful?

Discuss · then advance

Checkpoint 4 of 5

What does established match, and why is it not stateful?

TCP segments with the ACK or RST flag set — replies within a conversation somebody inside already started. It is not stateful because it inspects *flags* on each packet independently and keeps no record of connections; a crafted packet with ACK set will match.

CHECKPOINT 5 of 5

Name three supporting protocols that a restrictive inbound ACL must explicitly permit, and give the symptom for each if it is missed.

Discuss · then advance

Checkpoint 5 of 5

Name three supporting protocols that a restrictive inbound ACL must explicitly permit, and give the symptom for each if it is missed.

DHCP — without it hosts get no address and the VLAN looks completely dead. **DNS** — everything works by IP and nothing by name. **ICMP** — users and you lose the ability to test anything, and troubleshooting becomes guesswork.

What to take away

- Top down, first match wins, implicit `deny any` at the end. Order is behaviour.
- Wildcard mask = 255 minus each octet of the subnet mask. Check it by adding it to the network address.
- Standard (1–99, 1300–1999) matches source only — place near the destination. Extended (100–199, 2000–2699) matches source, destination, protocol and port — place near the source.
- Named ACLs with sequence numbers can be edited a line at a time. Numbered ones on older IOS cannot, and deleting one that is still applied denies everything.
- Apply with `ip access-group in|out` on an interface, or `access-class in` on the VTY lines.

What to take away (continued)

- `show access-lists` match counters are the real diagnostic: a zero on the last line means everything is being dropped; a zero in the middle means a shadowed rule.
- Remember DHCP, DNS and ICMP, or a restrictive ACL makes a VLAN look dead.

Where we got to

- Top down, first match wins, implicit deny any at the end. Order is behaviour.
- Wildcard mask = 255 minus each octet of the subnet mask. Check it by adding it to the network address.
- Standard (1–99, 1300–1999) matches source only — place near the destination. Extended (100–199, 2000–2699) matches source, destination, protocol and port — place near the source.
- Named ACLs with sequence numbers can be edited a line at a time. Numbered ones on older IOS cannot, and deleting one that is still applied denies everything.

NEXT

Chapter 25 — Layer 2 Security